1. 1. Les concepts clés et types de bases de données

1.1.Le modèle Relationnel (RDBMS)

- **Définition :** Modèle tabulaire où les données sont organisées en lignes et en colonnes (ex: MySQL, PostgreSQL, Oracle, MariaDB). Difficile à scaler horizontalement.
- **Usage idéal :** Applications nécessitant des transactions complexes, une forte consistance et tout ce qui touche à la finance/argent.

1.2.Le modèle NoSQL

- **Clé-valeur :** Optimisé pour la rapidité. Fonctionne en mémoire (RAM). Idéal pour les caches, le temps réel et les systèmes de files d'attente (ex: Redis).
- **Documentaire :** Stocke des objets JSON, offre une grande flexibilité sans structure rigide (ex: MongoDB pour les commentaires ou schémas évolutifs).
- **Orienté colonnes :** Stocke les données par colonne pour éviter de lire tout le disque. Offre une forte scalabilité horizontale pour les gros volumes, les statistiques et les agrégations (ex: Cassandra).
- **Graphe :** Excelle dans la gestion et le parcours des relations complexes et interconnectées (ex: Neo4j pour les réseaux sociaux ou cartographies).
- **Séries Temporelles (TSDB) :** Spécialisé pour les données de monitoring ou capteurs qui évoluent dans le temps, optimisé pour l'affichage de graphiques sans modification des données existantes (ex: InfluxDB, Prometheus).
- **Vectorielles :** Stocke des listes de nombres (vecteurs) pour calculer des distances entre concepts. Idéal pour l'IA et les moteurs de recherche intelligents (ex: Pinecone, Qdrant).

2. Le Théorème CAP

Un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes :

1. **Consistency (Cohérence - C) :** Tous les nœuds voient les mêmes données au même moment.
2. **Availability (Disponibilité - A) :** Chaque requête reçoit une réponse, même en cas de panne.
3. **Partition Tolerance (Tolérance aux partitions - P) :** Le système fonctionne malgré des pannes réseau.

- **Modèles courants :**
  - **CA :** MySQL non-distribué (privilégie la cohérence et la disponibilité locale).
  - **CP :** HBase (privilégie la cohérence stricte sur le réseau).
  - **AP :** Cassandra ou Redis distribué (privilégie la haute disponibilité et la résilience).

3. Méthodologie : Comment choisir son SGBD ?

Pour choisir le bon SGBD, il faut analyser méthodologiquement 4 critères :

1. **La nature des données :**
   - Données structurées ➔ Modèle Relationnel (SQL).
   - Données semi ou non structurées ➔ Modèle NoSQL.
2. **Les exigences de performance et scalabilité :**
   - Besoin de vitesse pure (temps réel) ➔ Base en mémoire (Clé-valeur).
   - Évolution massive des volumes ➔ Scalabilité horizontale (NoSQL Colonnes/Documents).
3. **Les besoins de disponibilité (Théorème CAP) :**
   - Tolérance aux pannes critique ➔ Systèmes distribués de type AP.
4. **Les contraintes budgétaires :**
   - Budget limité ➔ Privilégier les solutions Open Source performantes sur du matériel standard.

Exercice 1

Je recommande le modèle **NoSQL orienté documents** (comme **MongoDB**).

Vu le type de données que l'on veut stocker (non structurées), il est impossible d'utiliser un modèle relationnel sans perdre d'informations ; le **NoSQL** est donc bien meilleur dans ce cas. De plus, **MongoDB** offre une excellente flexibilité pour l'éventuelle croissance des données, s'adapte très bien au traitement en batch et c'est une solution open source, ce qui respecte la contrainte budgétaire.

Exercice 2

Je recommande une **architecture hybride NoSQL combinant le modèle Graphe (Neo4j) et le modèle Clé-valeur (Redis)**.

L'énoncé demande de modéliser les relations complexes entre les usines, les machines et les capteurs ; selon le cours, **Neo4j** excelle dans ce domaine. En parallèle, les données arrivent toutes les secondes, on utilise donc **Redis** car il fonctionne en mémoire et il est optimisé pour la rapidité d'accès en temps réel (AP du théorème CAP).

Exercice 3

Je recommande le modèle **NoSQL orienté colonnes** (comme **Cassandra**).

Les logs sont des données semi-structurées et volumineuses. Selon le cours, le **NoSQL** est parfaitement adapté à ce format et **Cassandra** offre une scalabilité horizontale pour les gros volumes. Ce modèle s'intègre parfaitement avec **Apache Spark** pour répondre au besoin d'analyses en temps réel et en temps différé (batch). De plus, **Cassandra** est un système distribué de type AP du théorème CAP, ce qui garantit que l'enregistrement des logs ne sera jamais interrompu.

Exercice 4

Je recommande le modèle **NoSQL** de type AP (comme **Cassandra** ou **Redis** pour la rapidité).

Les données sont alimentées en continu par API et demandent un temps de réponse inférieur à une seconde. Selon le cours, certaines bases fonctionnent entièrement en mémoire (comme **Redis**) pour des temps de réponse ultra-rapides, et les bases distribuées offrent une haute disponibilité face aux flux continus (AP du théorème CAP).